Finding Time Complexity of Algorithms

1. Problem: Finding Complexity Using Counter Method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function(int n)
{
    int i = 1;
    int s = 1;

    while (s <= n) {
        i++;
        s += i;
    }
}
```

Note

No counter increment is needed for declarations, `scanf()`, or `printf()` statements.

Input

- A positive integer `n`

Output

- Print the value of the counter variable

C Program

```
#include <stdio.h>

int main() {
    int n, count = 0;
    scanf("%d", &n);

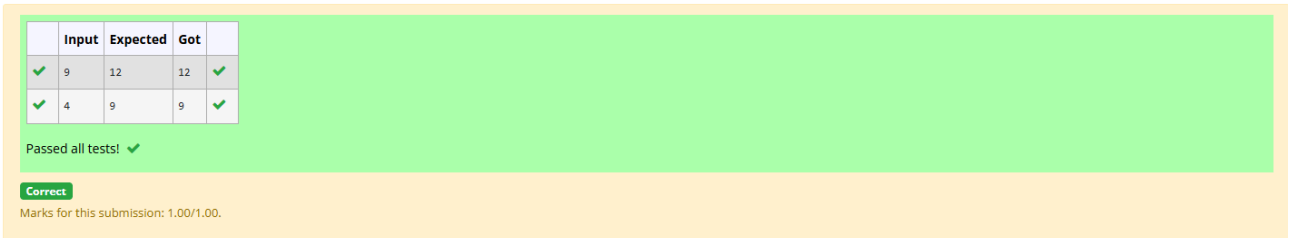
    int i = 1;
    count++;
    int s = 1;
    count++;

    while (count++, s <= n) {
        i++;
        count++;

        s += i;
```

```
            count++;
        }

        printf("%d", count);
        return 0;
    }
```



	Input	Expected	Got	
✔	9	12	12	✔
✔	4	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

2. Problem: Finding Complexity Using Counter Method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if (n == 1) {
        printf("*");
    } else {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                printf("*");
                printf("*");
                break;
            }
        }
    }
}
```

Note

No counter increment is needed for declarations, `scanf()`, or `printf()` statements.

Input

- A positive integer `n`

Output

- Print the value of the counter variable

C Program

```c
#include <stdio.h>

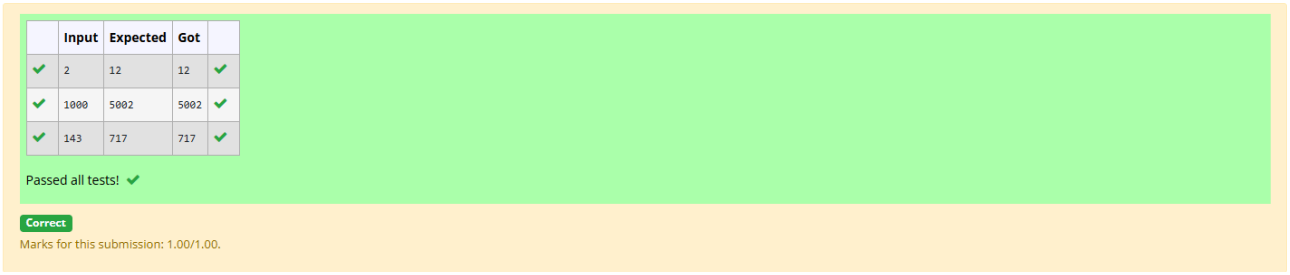
int main() {
    int n, count = 0;
    scanf("%d", &n);

    if (n == 1) {
        count++;
    } else {
        int i = 1;
        count++;

        while (count++, i <= n) {
            count++;
            count++;
            count++;

            i++;
            count++;
        }
    }

    printf("%d", count);
    return 0;
}
```



	Input	Expected	Got	
✔	2	12	12	✔
✔	1000	5002	5002	✔
✔	143	717	717	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

3. Problem: Finding Complexity Using Counter Method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
Factor(num) {
    for (i = 1; i <= num; ++i) {
        if (num % i == 0) {
            printf("%d ", i);
        }
    }
}
```

Note

No counter increment is needed for declarations, `scanf()`, or `printf()` statements.

Input

- A positive integer `n`

Output

- Print the value of the counter variable

C Program

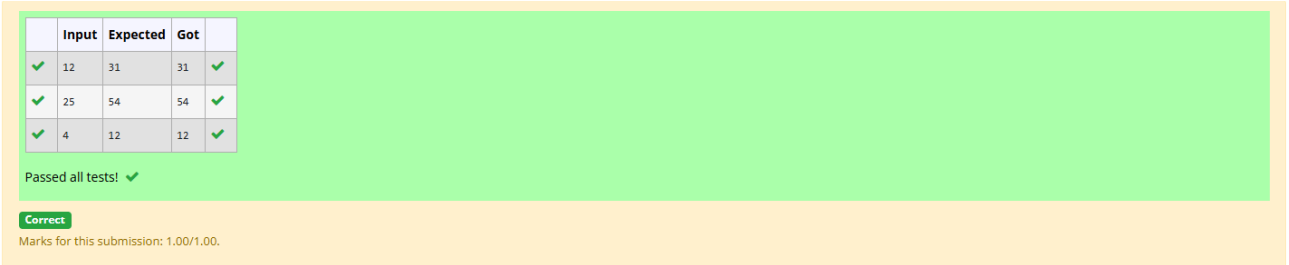
```c
#include <stdio.h>

int main() {
    int count = 0;
    count++;

    int num;
    scanf("%d", &num);

    for (int i = 1; i <= num; i++) {
        if (num % i == 0) {
            count++;
        }
        count += 2;
    }

    printf("%d", count);
    return 0;
}
```



	Input	Expected	Got	
✔	12	31	31	✔
✔	25	54	54	✔
✔	4	12	12	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

4. Problem: Finding Complexity Using Counter Method

Convert the following algorithm into a program and find its time complexity using the counter method.

```c
void function(int n)
{
    int c = 0;

    for (int i = n / 2; i < n; i++)
```

```
        for (int j = 1; j < n; j = 2 * j)
            for (int k = 1; k < n; k = k * 2)
                c++;
}
```

Note

No counter increment is needed for declarations, `scanf()`, or `printf()` statements.

Input

- A positive integer `n`

Output

- Print the value of the counter variable

C Program

```c
#include <stdio.h>

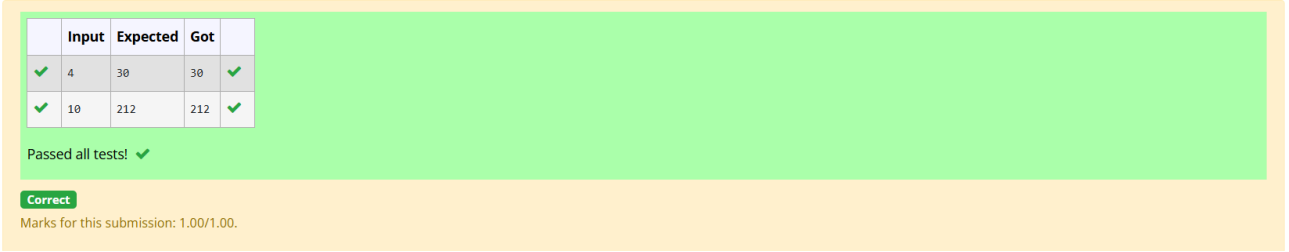
int main() {
    int count = 0, n;
    scanf("%d", &n);
    count++;

    for (int i = n / 2; i < n; i++) {
        count += 2;

        for (int j = 1; j < n; j = 2 * j) {
            count += 2;

            for (int k = 1; k < n; k = k * 2) {
                count += 2;
            }
        }
    }

    count++;
    printf("%d", count);
    return 0;
}
```



	Input	Expected	Got	
✔	4	30	30	✔
✔	10	212	212	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5. Problem: Finding Complexity Using Counter Method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void reverse(int n)
{
    int rev = 0, remainder;

    while (n != 0) {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n /= 10;
    }

    print(rev);
}
```

Note

No counter increment is needed for declarations, `scanf()`, or `printf()` statements.

Input

- A positive integer `n`

Output

- Print the value of the counter variable

C Program

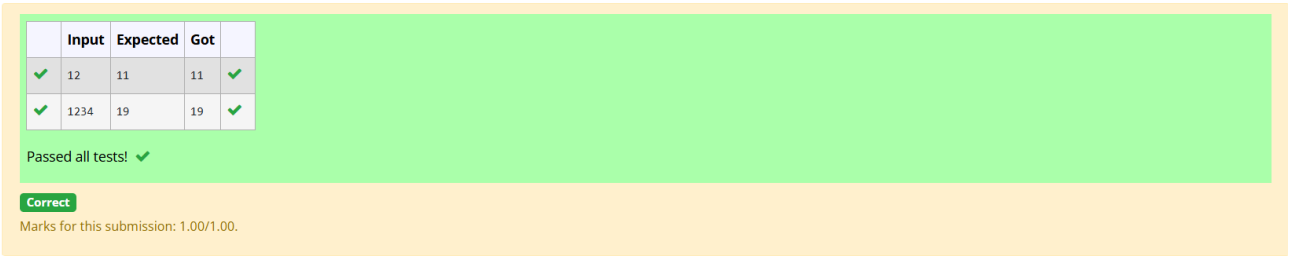
```c
#include <stdio.h>

int main() {
    int n, count = 2;
    int rev = 0, remainder;
    scanf("%d", &n);

    while (count++, n != 0) {
        remainder = n % 10;
        rev = rev * 10 + remainder;
```

```
        n /= 10;
        count += 3;
    }

    printf("%d", count);
    return 0;
}
```



	Input	Expected	Got	
✔	12	11	11	✔
✔	1234	19	19	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Greedy Algorithm

1. G - Coin Problem

Problem

Write a program to take a value `V` and make change for `V` rupees using the minimum number of coins/notes. Assume an infinite supply of Indian currency denominations:

`{1, 2, 5, 10, 20, 50, 100, 500, 1000}`.

Input Format

- Take an integer from standard input.

Output Format

- Print the minimum number of coins/notes needed to make the change.

Example

Input:

```
64
```

Output:

```
4
```

Explanation

We need:

- one `50` rupee note
- one `10` rupee note
- two `2` rupee coins

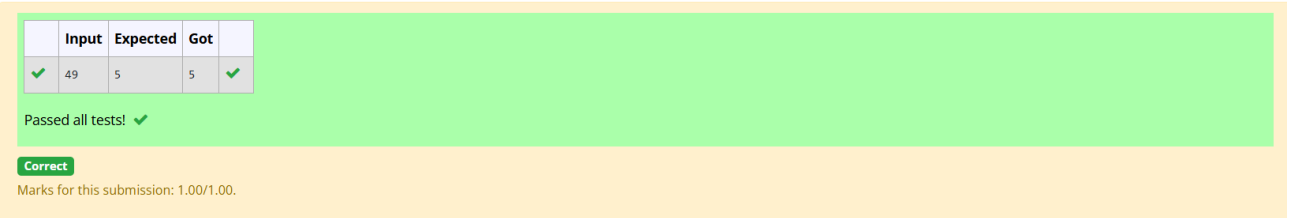
C Program

```c
#include <stdio.h>

int main() {
    int arr[9] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int n;
    scanf("%d", &n);

    int c = 0;
    for (int i = 0; i < 9; i++) {
        if (n >= arr[i]) {
            c += n / arr[i];
            n %= arr[i];
        }
    }

    printf("%d", c);
    return 0;
}
```



2. G - Cookies Problem

Problem

Assume you are a parent who wants to give cookies to children. Each child can receive at most one cookie.

Each child `i` has a greed factor `g[i]`, which is the minimum cookie size that makes them content. Each cookie `j` has a size `s[j]`. If `s[j] >= g[i]`, the child can be satisfied.

Your goal is to maximize the number of satisfied children.

Example 1

Input:

```
3
1 2 3
2
1 1
```

Output:

```
1
```

Explanation

There are 3 children with greed factors `1, 2, 3` and 2 cookies of size `1` each.

Only the child with greed factor `1` can be satisfied.

Constraints

- `1 <= g.length <= 3 * 10^4`
- `0 <= s.length <= 3 * 10^4`
- `1 <= g[i], s[j] <= 2^31 - 1`

C Program

```c
#include <stdio.h>

int main() {
    int n, m;

    if (scanf("%d", &n) != 1) return 0;
    int g[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &g[i]);
    }

    if (scanf("%d", &m) != 1) return 0;
    int s[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &s[i]);
    }

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (g[j] > g[j + 1]) {
                int temp = g[j];
                g[j] = g[j + 1];
                g[j + 1] = temp;
            }
```

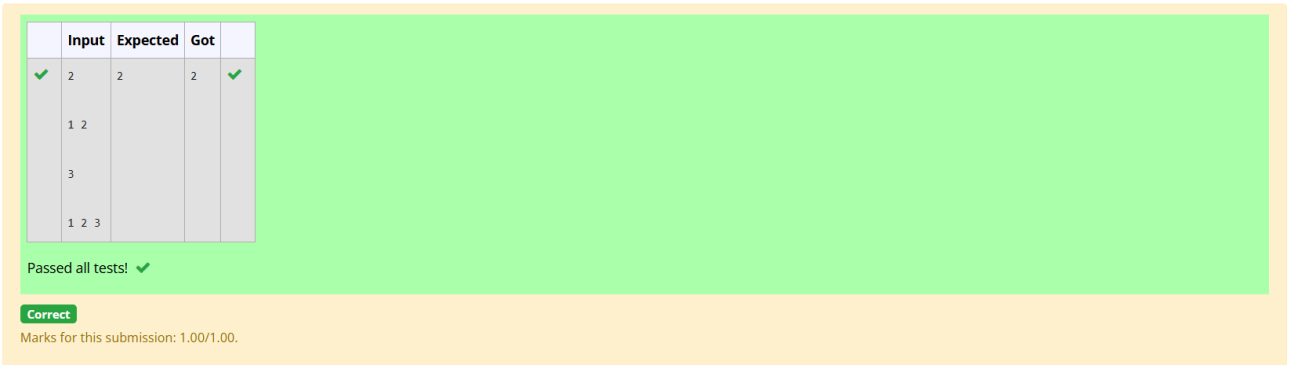
```
        }
    }

    for (int i = 0; i < m - 1; i++) {
        for (int j = 0; j < m - i - 1; j++) {
            if (s[j] > s[j + 1]) {
                int temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }

    int child_ptr = 0;
    int cookie_ptr = 0;

    while (child_ptr < n && cookie_ptr < m) {
        if (s[cookie_ptr] >= g[child_ptr]) {
            child_ptr++;
        }
        cookie_ptr++;
    }

    printf("%d\n", child_ptr);
    return 0;
}
```



3. G - Burger Problem

Problem

A person needs to eat burgers. Each burger contains a certain number of calories. After eating a burger, the person must run a distance to burn the calories.

If he has eaten `i` burgers with `c` calories each, then he has to run at least `3i * c` kilometers to burn them out.

For example, if he ate 3 burgers with calories in the order `[1, 3, 2]`, then the kilometers he needs to run are:

- `(3 * 1) * 1 + (3 * 3) * 2 + (3 * 2) * 3 = 1 + 9 + 18 = 28`

This is not the minimum; we need to try all orders and choose the minimum value.

Input Format

- The first line contains the number of burgers.
- The second line contains the calories of each burger as `n` space-separated integers.

Output Format

- Print the minimum number of kilometers needed to run to burn the calories.

Sample Input

```
3
5 10 7
```

Sample Output

```
76
```

C Program

```c
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

int main() {
    int n;
    scanf("%d", &n);

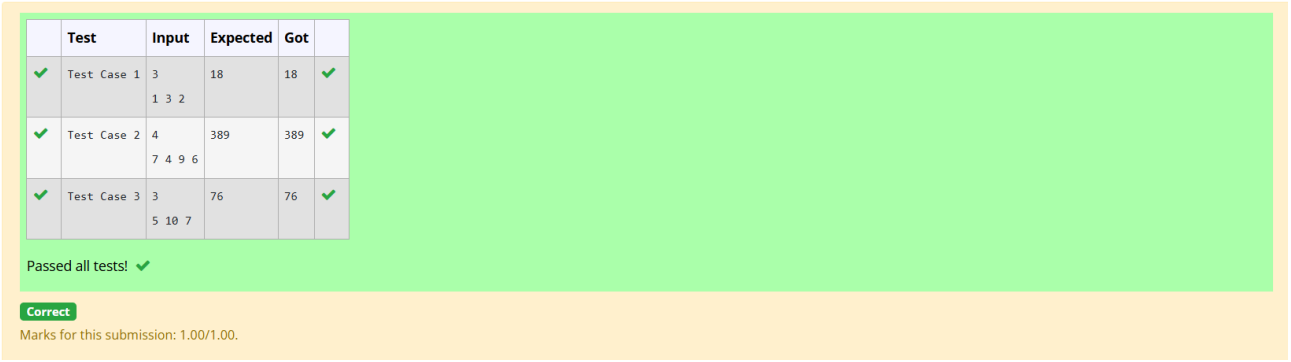
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] < arr[j]) {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    int run = 0;
```

```
    for (int i = 0; i < n; i++) {
        run += pow(n, i) * arr[i];
    }

    printf("%d", run);
    return 0;
}
```



	Test	Input	Expected	Got	
✔	Test Case 1	3 1 3 2	18	18	✔
✔	Test Case 2	4 7 4 9 6	389	389	✔
✔	Test Case 3	3 5 10 7	76	76	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

4. G - Array Sum Max Problem

Problem

Given an array of `N` integers, maximize the sum of `arr[i] * i`, where `i` is the index of the element (`i = 0, 1, 2, ..., N`).

Write an algorithm based on the greedy technique with complexity `O(n log n)`.

Input Format

- The first line specifies the number of elements `n`.
- The next `n` lines contain the array elements.

Output Format

- Print the maximum array sum.

Sample Input

```
5
2 5 3 4 0
```

Sample Output

```
40
```

C Program

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];
    int s = 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (a[i] < a[j]) {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }

    for (int i = 0; i < n; i++) {
        s += a[i] * i;
    }

    printf("%d", s);
    return 0;
}
```

	Input	Expected	Got	
✔	5 2 5 3 4 0	40	40	✔
✔	10 2 2 2 4 4 3 3 5 5 5	191	191	✔
✔	2 45 3	45	45	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5. G - Product of Array Elements - Minimum

Problem

Given two arrays, `array_One[]` and `array_Two[]`, of the same size `N`, rearrange the arrays so that the sum of products of corresponding pairs is minimum.

That is, minimize:

`SUM(A[i] * B[i])` for all `i`.

Example

Input:

```
3
1
2
3
4
5
6
```

Result:

```
28
```

C Program

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];
    int b[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (int i = 0; i < n; i++) {
        scanf("%d", &b[i]);
    }

    for (int i = 0; i < n; i++) {
```

```
            for (int j = i + 1; j < n; j++) {
                if (a[i] > a[j]) {
                    int temp = a[i];
                    a[i] = a[j];
                    a[j] = temp;
                }
            }
        }

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (b[i] < b[j]) {
                    int temp = b[i];
                    b[i] = b[j];
                    b[j] = temp;
                }
            }
        }

        int s = 0;
        for (int i = 0; i < n; i++) {
            s += a[i] * b[i];
        }

        printf("%d", s);
        return 0;
    }
```

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔

Divide and Conquer

1. Number of Zeros in a Given Array

Problem Statement

Given an array of `1`s and `0`s that has all `1`s first, followed by all `0`s, find the number of zeros using divide and conquer.

Input Format

- First line: integer `m` — size of the array
- Next `m` lines: array elements

Output Format

- First line: integer — number of zeroes present in the array

C Program

```c
#include <stdio.h>

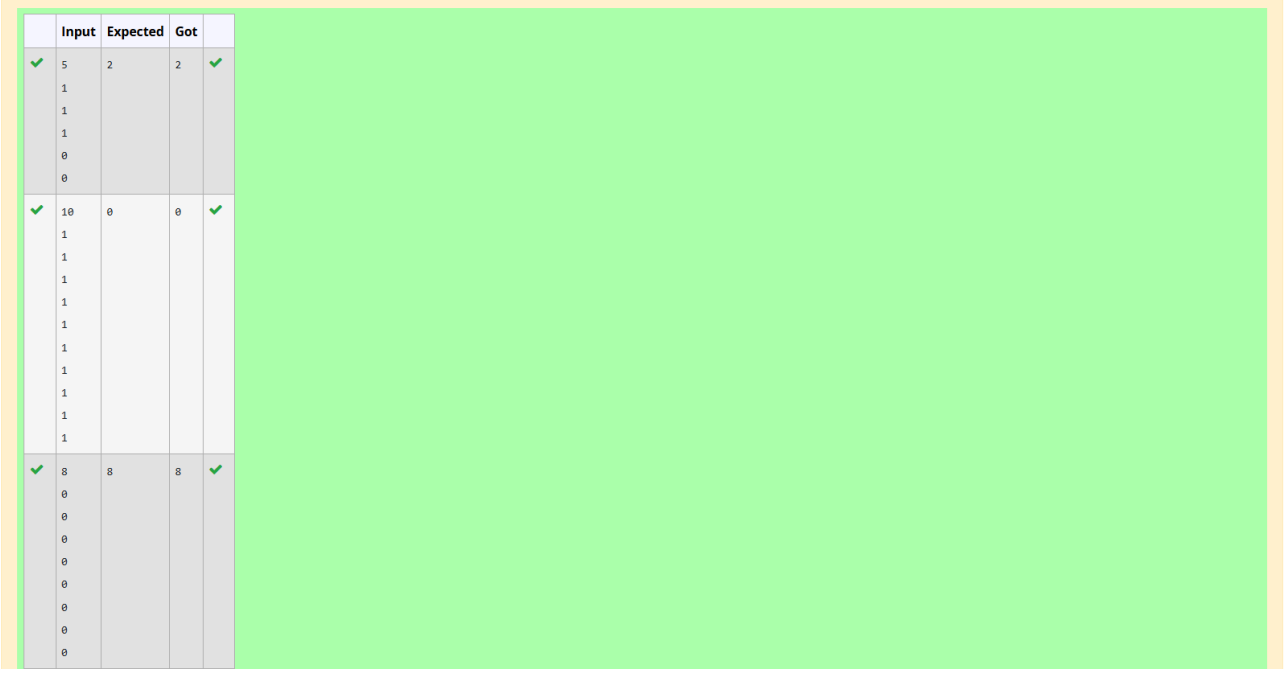
int countZeros(int arr[], int low, int high) {
    if (low > high) {
        return 0;
    }
    if (arr[low] == 0) {
        return (high - low + 1);
    }
    if (arr[high] == 1) {
        return 0;
    }

    int mid = low + (high - low) / 2;
    return countZeros(arr, low, mid) + countZeros(arr, mid + 1, high);
}

int main() {
    int m;
    scanf("%d", &m);

    int arr[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }

    printf("%d", countZeros(arr, 0, m - 1));
    return 0;
}
```



	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔

2. Majority Element

Problem Statement

Given an array of size `n`, return the majority element.

The majority element is the element that appears more than `floor(n / 2)` times. You may assume the majority element always exists.

Example 1

Input:

```
nums = [3, 2, 3]
```

Output:

```
3
```

Example 2

Input:

```
nums = [2, 2, 1, 1, 1, 2, 2]
```

Output:

```
2
```

Constraints

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

Example Table

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

C Program

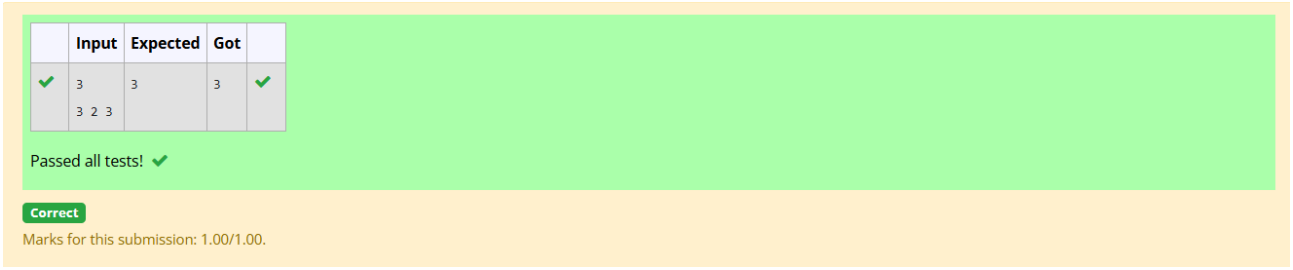
```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int c = 1, find = arr[0];
    for (int i = 0; i < n; i++) {
        if (c == 0) {
            find = arr[i];
            c = 1;
        } else if (arr[i] == find) {
            c++;
        } else {
            c--;
        }
    }

    printf("%d", find);
    return 0;
}
```



3. Finding Floor Value

Problem Statement

Given a sorted array and a value x, the floor of x is the largest element in the array smaller than or equal to x. Write a divide-and-conquer algorithm to find the floor of x.

Input Format

- First line: integer n — size of the array
- Next n lines: array elements
- Last line: integer x

Output Format

- First line: integer — floor value for x

C Program

```c
#include <stdio.h>

int findFloor(int arr[], int low, int high, int x) {
    if (low > high) {
        return -1;
    }
    if (x >= arr[high]) {
        return arr[high];
    }

    int mid = (low + high) / 2;

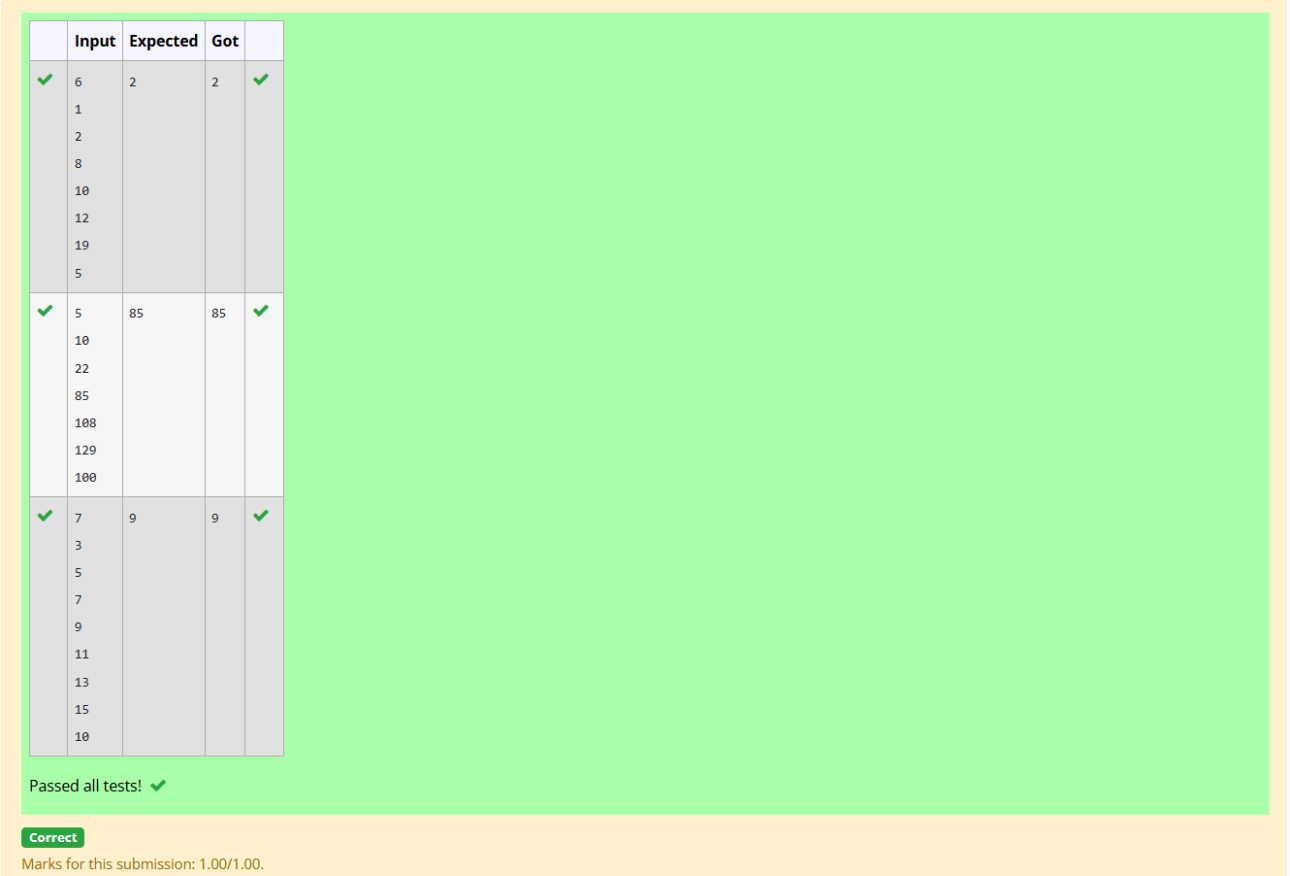
    if (arr[mid] == x) {
        return arr[mid];
    }
    if (mid > 0 && arr[mid - 1] <= x && x < arr[mid]) {
        return arr[mid - 1];
    }
    if (x < arr[mid]) {
        return findFloor(arr, low, mid - 1, x);
    }

    return findFloor(arr, mid + 1, high, x);
}
```

```c
int main() {
    int n, x;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);
    printf("%d", findFloor(arr, 0, n - 1, x));
    return 0;
}
```



	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

4. Two Elements Sum to x

Problem Statement

Given a sorted array of integers `arr[]` and a number `x`, write a recursive program using divide and conquer to check whether there exist two elements whose sum is `x`. If such elements exist, print them; otherwise print `No`.

Input Format

- First line: integer `n` — size of the array
- Next `n` lines: array elements

- Last line: integer `x` — sum value

Output Format

- First line: element 1
- Second line: element 2

C Program

```c
#include <stdio.h>

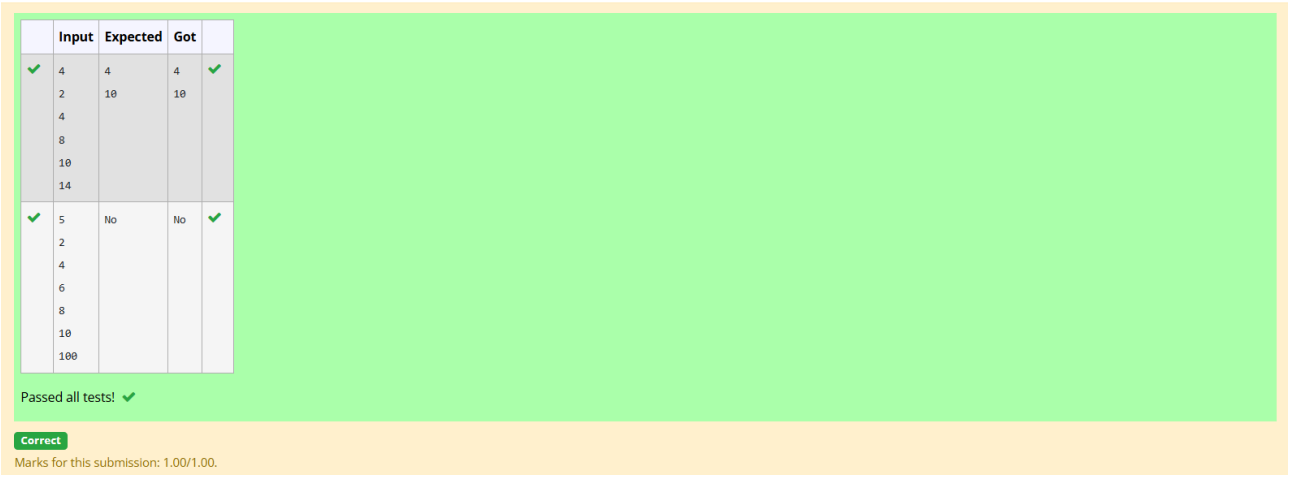
int findPair(int arr[], int left, int right, int x) {
    if (left >= right) {
        printf("No");
        return 0;
    }

    int sum = arr[left] + arr[right];
    if (sum == x) {
        printf("%d\n%d", arr[left], arr[right]);
        return 1;
    } else if (sum < x) {
        return findPair(arr, left + 1, right, x);
    } else {
        return findPair(arr, left, right - 1, x);
    }
}

int main() {
    int n, x;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);
    findPair(arr, 0, n - 1, x);
    return 0;
}
```



	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5. Implementation of Quick Sort

Problem

Write a program to implement the quick sort algorithm.

Input Format

- First line: integer n — number of elements
- Next n lines: elements of the list

Output

- Sorted list of elements

Example Table

Input	Result
5 67 34 12 98 78	12 34 67 78 98

C Program

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
```

```
            if (arr[j] < pivot) {
                i++;
                swap(&arr[i], &arr[j]);
            }
        }

        swap(&arr[i + 1], &arr[high]);
        return i + 1;
    }

    void quickSort(int arr[], int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    int main() {
        int n;
        scanf("%d", &n);

        int arr[n];
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }

        quickSort(arr, 0, n - 1);
        for (int i = 0; i < n; i++) {
            printf("%d ", arr[i]);
        }

        return 0;
    }
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Dynamic Programming

1. DP - Playing with Numbers

Problem

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram's turn, so he gave Sita a positive integer `n` and two numbers `1` and `3`. He asked her to find the number of ways by which `n` can be represented using `1` and `3`.

Example

Input: `6`

Output: `6`

Explanation:

There are 6 ways to represent 6 using 1 and 3:

- `1 + 1 + 1 + 1 + 1 + 1`
- `3 + 3`
- `1 + 1 + 1 + 3`
- `1 + 1 + 3 + 1`
- `1 + 3 + 1 + 1`
- `3 + 1 + 1 + 1`

Input Format

- The first line contains the number `n`.

Output Format

- Print the number of possible ways `n` can be represented using `1` and `3`.

Sample Input

```
6
```

Sample Output

```
6
```

C Program

```c
#include <stdio.h>

long long count(int n) {
    if (n < 0) return 0;
    if (n == 0 || n == 1 || n == 2) return 1;
```

```
    long long dp[n + 1];
    dp[0] = 1;
    dp[1] = 1;
    dp[2] = 1;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 3];
    }

    return dp[n];
}

int main() {
    int n;
    scanf("%d", &n);
    printf("%lld", count(n));
    return 0;
}
```

	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

2. DP - Playing with Chessboard

Problem

Ram is given an `n x n` chessboard, with each cell having a monetary value. Ram stands at cell `(0, 0)`, the top-left white rook. He must reach the bottom-right black rook position `(n - 1, n - 1)` while moving only one step right or one step down at a time. The goal is to find the path with the maximum monetary value.

Example

Input:

```
3
1 2 4
2 3 4
8 7 1
```

Output:

```
19
```

Explanation

There are 6 possible paths, and the optimal path value is:

`1 + 2 + 8 + 7 + 1 = 19`

Input Format

- The first line contains the integer `n`.
- The next `n` lines contain the values of the `n x n` chessboard.

Output Format

- Print the maximum monetary value of the path.

C Program

```c
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n][n];
    int brr[n][n];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &arr[i][j]);
        }
    }

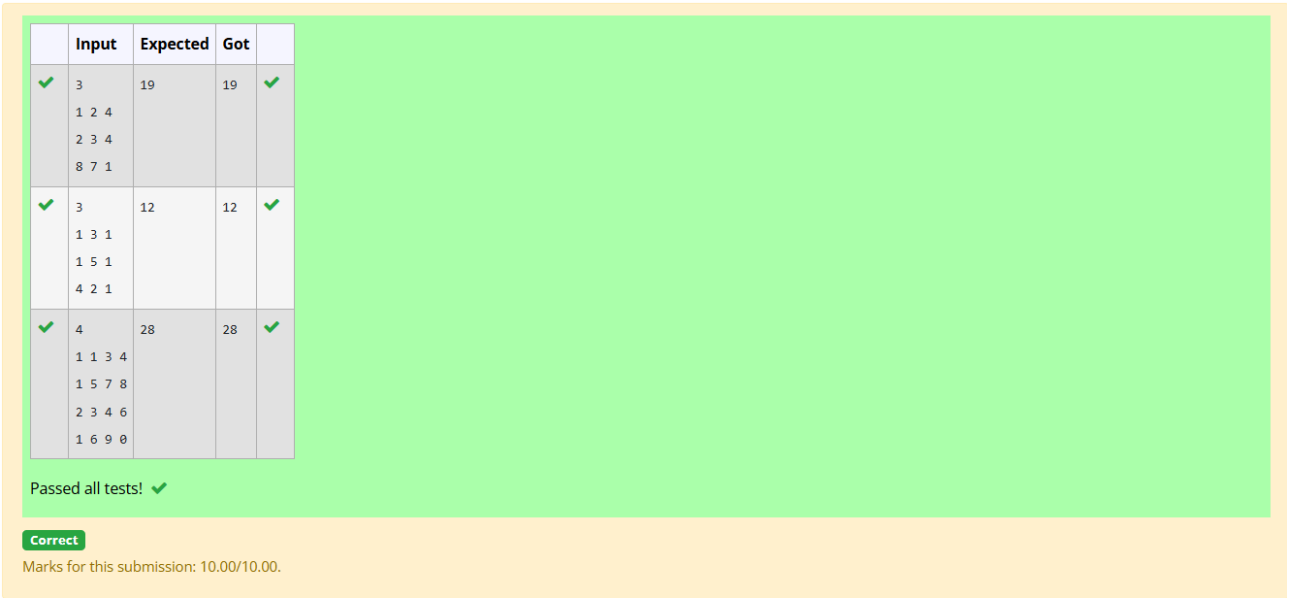
    brr[0][0] = arr[0][0];
    for (int i = 1; i < n; i++) {
        brr[i][0] = brr[i - 1][0] + arr[i][0];
    }

    for (int j = 1; j < n; j++) {
        brr[0][j] = brr[0][j - 1] + arr[0][j];
    }

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
```

```
            brr[i][j] = arr[i][j] + max(brr[i - 1][j], brr[i][j - 1]);
        }
    }

    printf("%d\n", brr[n - 1][n - 1]);
    return 0;
}
```



	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

3. DP - Longest Common Subsequence

Problem

Given two strings, find the length of the longest common subsequence (not necessarily contiguous) between them.

Example

```
s1: ggtabe
s2: tgatasb
```

The length is 4.

Explanation

This is solved using dynamic programming.

Example Table

Input	Result
aab azb	2

C Program

```c
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    char a[1000], b[1000];
    scanf("%s %s", a, b);

    int n = strlen(a);
    int m = strlen(b);
    int dp[n + 1][m + 1];

    for (int i = 0; i <= n; i++) {
        dp[i][0] = 0;
    }

    for (int i = 0; i <= m; i++) {
        dp[0][i] = 0;
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    printf("%d", dp[n][m]);
    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

4. DP - Longest Non-decreasing Subsequence

Problem

Find the length of the longest non-decreasing subsequence in a given sequence.

Example

Input:

```
9
-1 3 4 5 2 2 2 2 3
```

The subsequence is:

```
[-1, 2, 2, 2, 2, 3]
```

Output:

```
6
```

C Program

```c
#include <stdio.h>

int max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    int dp[n];

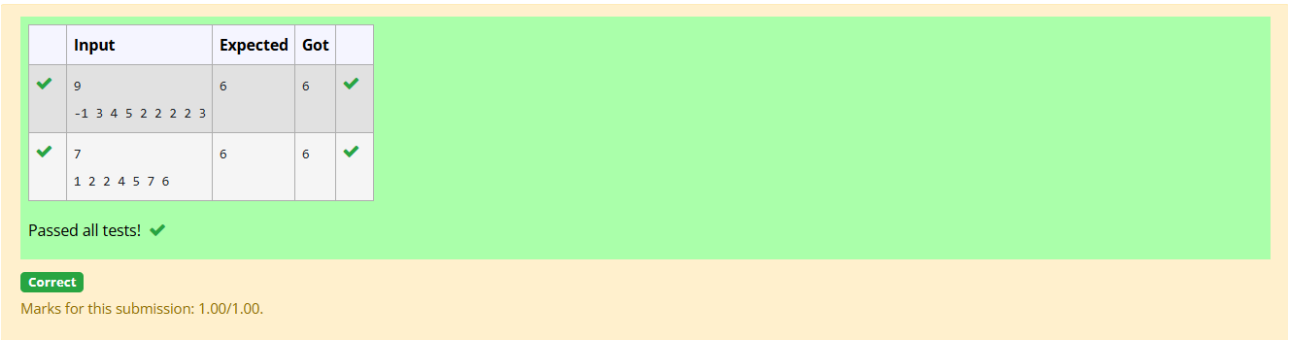
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        dp[i] = 1;
    }

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (arr[j] <= arr[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
```

```
    }

    int ans = dp[0];
    for (int i = 1; i < n; i++) {
        ans = max(ans, dp[i]);
    }

    printf("%d", ans);
    return 0;
}
```



	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Competitive Programming

1. Finding Duplicates (O(n^2) Time, O(1) Space)

Problem

Find duplicate in array.

Given a read-only array of n integers between 1 and n, find one number that repeats.

Input Format

- The first line contains the number of elements.
- The next n lines contain the array elements.

Output Format

- Print the repeated element x.

Example

Input	Result
5 1 1 2 3 4	1

C Program

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int slow = arr[0];
    int fast = arr[0];

    do {
        slow = arr[slow];
        fast = arr[arr[fast]];
    } while (slow != fast);

    slow = arr[0];
    while (slow != fast) {
        slow = arr[slow];
        fast = arr[fast];
    }

    printf("%d", slow);
    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

2. Finding Duplicates (O(n) Time, O(1) Space)

Problem

Find duplicate in array.

Given a read-only array of `n` integers between `1` and `n`, find one number that repeats.

Input Format

- The first line contains the number of elements.

- The next `n` lines contain the array elements.

Output Format

- Print the repeated element `x`.

Example

Input	Result
5 1 1 2 3 4	1

C Program

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    scanf("%d", &n);

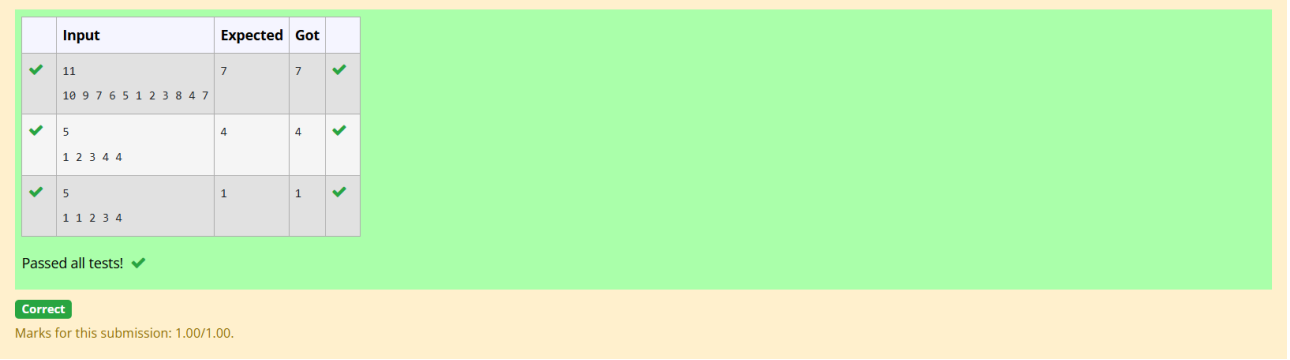
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int slow = arr[0];
    int fast = arr[0];

    do {
        slow = arr[slow];
        fast = arr[arr[fast]];
    } while (slow != fast);

    slow = arr[0];
    while (slow != fast) {
        slow = arr[slow];
        fast = arr[fast];
    }

    printf("%d", slow);
    return 0;
}
```



	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

3. Print Intersection of Two Sorted Arrays (O(m*n) Time, O(1) Space)

Problem

Find the intersection of two sorted arrays.

In other words, given two sorted arrays, find all elements that occur in both arrays.

Input Format

- The first line contains `T`, the number of test cases.
- For each test case:
    1. The first line contains `N1`, followed by `N1` integers of the first array.
    2. The second line contains `N2`, followed by `N2` integers of the second array.

Output Format

- Print the intersection of the arrays in a single line.

Example

Input:

```
1
3 10 17 57
6 2 7 10 15 57 246
```

Output:

```
10 57
```

Input:

```
1
6 1 2 3 4 5 6
```

```
2 1 6
```

Output:

```
1 6
```

Example Table

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

C Program

```c
#include <stdio.h>

int main() {
    int t;
    scanf("%d", &t);

    while (t--) {
        int n, m;
        scanf("%d", &n);

        int arr[n];
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }

        scanf("%d", &m);
        int brr[m];
        for (int i = 0; i < m; i++) {
            scanf("%d", &brr[i]);
        }

        int f = 1;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (arr[i] == brr[j]) {
                    if (!f) {
                        printf(" ");
                    }
                    printf("%d", arr[i]);
                    f = 0;
```

```
                    break;
                }
            }
        }

        printf("\n");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

4. Print Intersection of Two Sorted Arrays (O(m*n) Time, O(1) Space)

Problem

Find the intersection of two sorted arrays.

In other words, given two sorted arrays, find all elements that occur in both arrays.

Input Format

- The first line contains `T`, the number of test cases.
- For each test case:
    1. The first line contains `N1`, followed by `N1` integers of the first array.
    2. The second line contains `N2`, followed by `N2` integers of the second array.

Output Format

- Print the intersection of the arrays in a single line.

Example

Input:

```
1
3 10 17 57
6 2 7 10 15 57 246
```

Output:

```
10 57
```

Input:

```
1
6 1 2 3 4 5 6
2 1 6
```

Output:

```
1 6
```

Example Table

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

C Program

```c
#include <stdio.h>

int main() {
    int t;
    scanf("%d", &t);

    while (t--) {
        int n, m;
        scanf("%d", &n);

        int arr[n];
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
```

```
        }

        scanf("%d", &m);
        int brr[m];
        for (int i = 0; i < m; i++) {
            scanf("%d", &brr[i]);
        }

        int f = 1;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (arr[i] == brr[j]) {
                    if (!f) {
                        printf(" ");
                    }
                    printf("%d", arr[i]);
                    f = 0;
                    break;
                }
            }
        }

        printf("\n");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5. Pair with Difference (O(n^2) Time, O(1) Space)

Problem

Given an array `A` of sorted integers and another non-negative integer `k`, find whether there exist two indices `i` and `j` such that:

`A[j] - A[i] = k`, where `i != j`.

Input Format

- The first line contains `n`, the number of elements in the array.
- The next `n` lines contain the array elements.
- The next line contains `k`, a non-negative integer.

Output Format

- Print `1` if the pair exists.
- Print `0` if no pair exists.

Explanation

YES, because `5 - 1 = 4`.

Example

Input	Result
3 1 3 5 4	1

C Program

```c
#include <stdio.h>

int main() {
    int n, t;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &t);
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[j] - arr[i] == t) {
                printf("1");
                return 0;
            }
        }
    }

    printf("0");
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

6. Pair with Difference (O(n^2) Time, O(1) Space)

Problem

Given an array `A` of sorted integers and another non-negative integer `k`, find whether there exist two indices `i` and `j` such that:

`A[j] - A[i] = k`, where `i != j`.

Input Format

- The first line contains `n`, the number of elements in the array.
- The next `n` lines contain the array elements.
- The next line contains `k`, a non-negative integer.

Output Format

- Print `1` if the pair exists.
- Print `0` if no pair exists.

Explanation

YES, because `5 - 1 = 4`.

Example

Input	Result
3 1 3 5 4	1

C Program

```c
#include <stdio.h>

int main() {
    int n, t;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &t);
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[j] - arr[i] == t) {
                printf("1");
                return 0;
            }
        }
    }

    printf("0");
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.